Singly Linked List – Delete At End And Time Complexity.

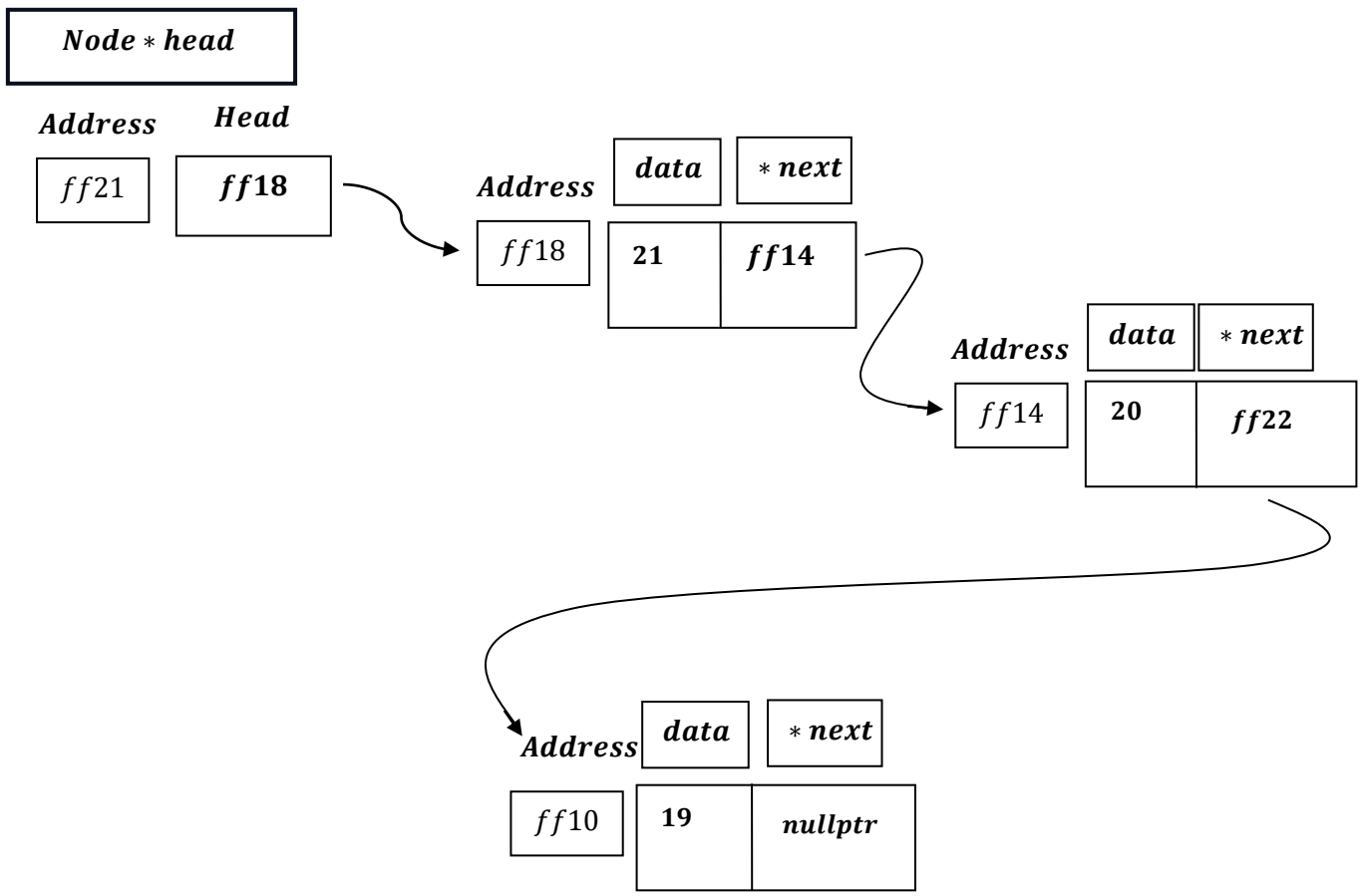
Program:

```
void deleteFromEnd(Node **head)
{
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        Node *previous = nullptr;

        while (current->next != nullptr)
        {
            previous = current;
            current = current->next;
        }
        if (previous == nullptr)
        {
            *head = nullptr;
        }
        else
        {
            previous->next = nullptr;
        }
        cout << "Deleted node: " << current->data << endl;
        free(current);
        current = nullptr;
        previous = nullptr;
        cout << "Node is deleted successfully." << endl;
    }
```

```
}

…..

 case 7:
        {
            cout << endl;
            cout << endl;
            deleteFromEnd(&head);
            break;
        }
```

Program Analysis:

We recall previous formed linked list:



At first we move to the function call:

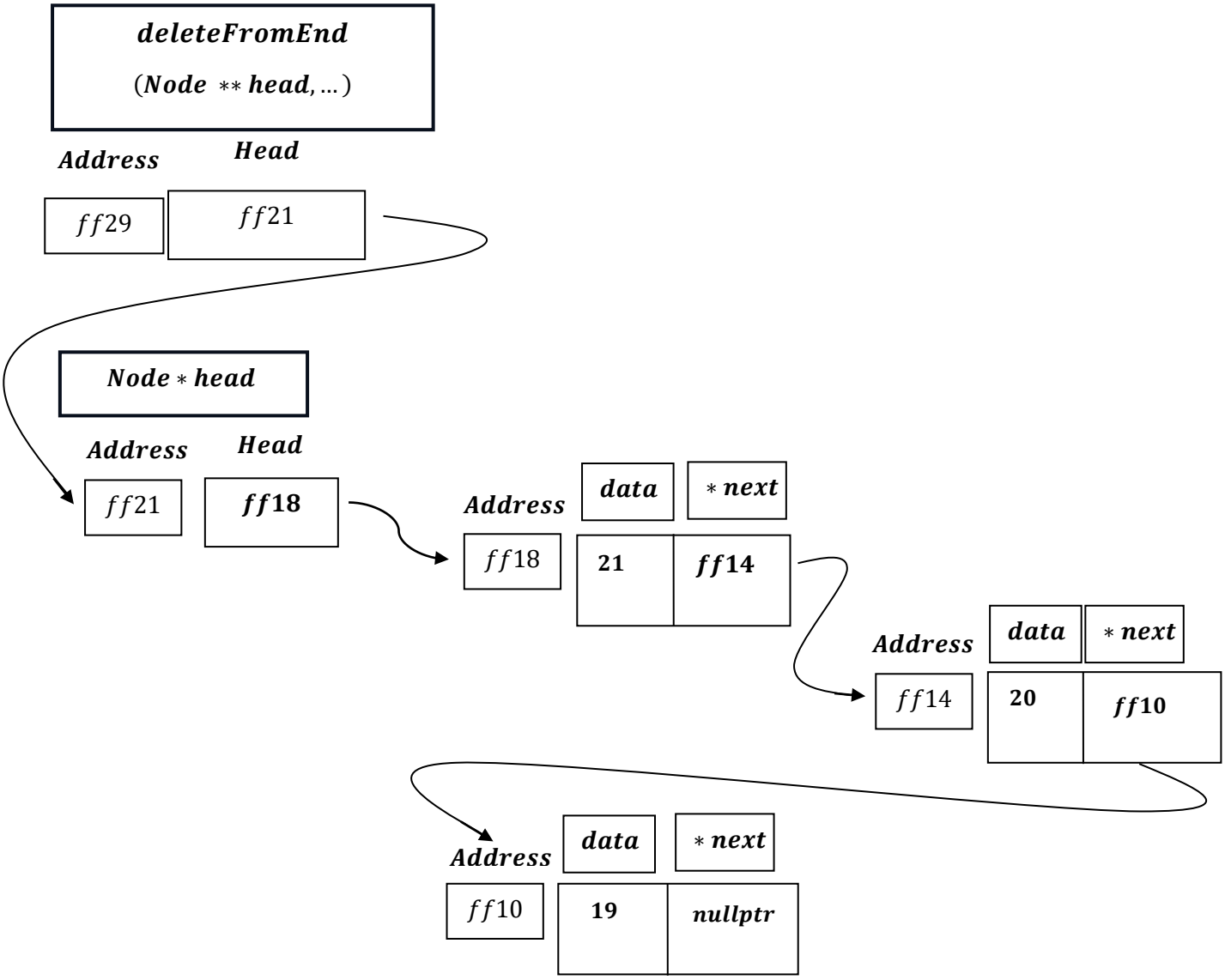
```
deleteFromEnd(&head);
```

Hence, at function call we pass:

$$deleteFromEnd(ff18); \rightarrow \textit{That is the address of the `head`}.$$

Next :

```
void deleteFromEnd(Node **head) {…}
```



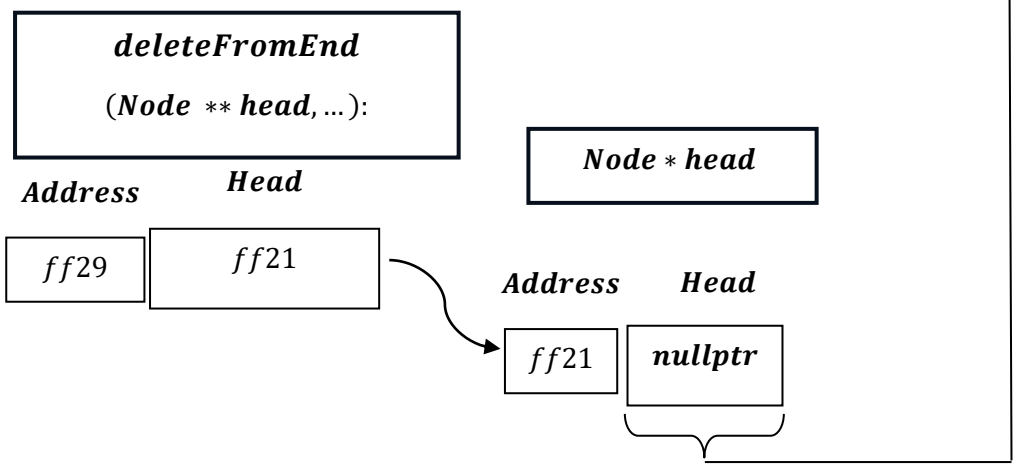
Now if Head pointer of Node structure points to nullptr then list is empty ,

i.e. Head carries value nullptr (not pointing to any valid memory location).

```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```

if(∗ head == nullptr) if it is true

If true then Why true ?
Ans: Though ∗ head which is of Node ∗∗ head
temporary pointer to pointer variable of the function
, as ∗ head is used , it dereference from Node ∗∗ head and
refer to Node ∗ head global pointer to which Node ∗∗ head
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the deleteFromEnd function.



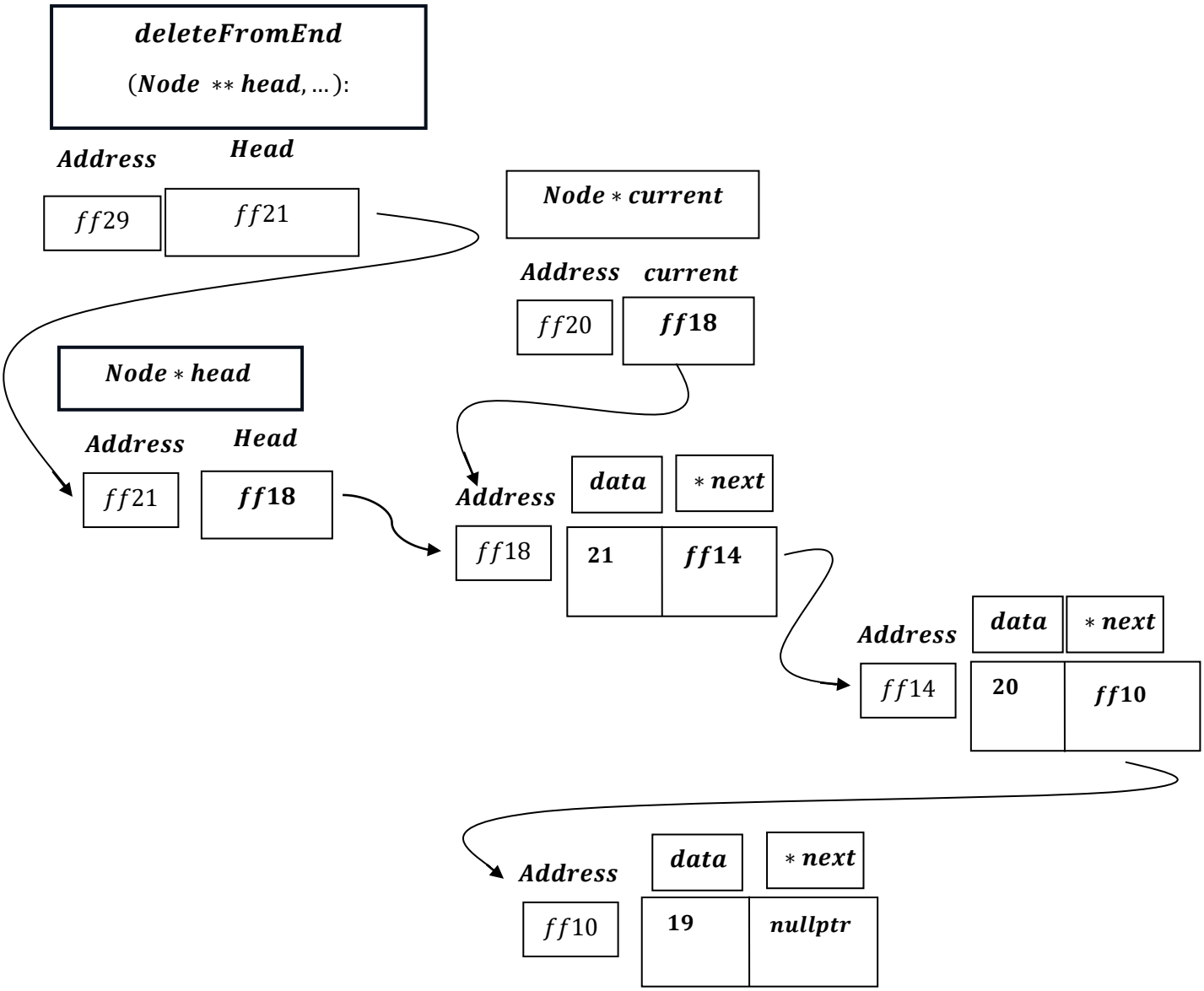
Then , it prints a message : `Error: List is empty.`

Next is else part:

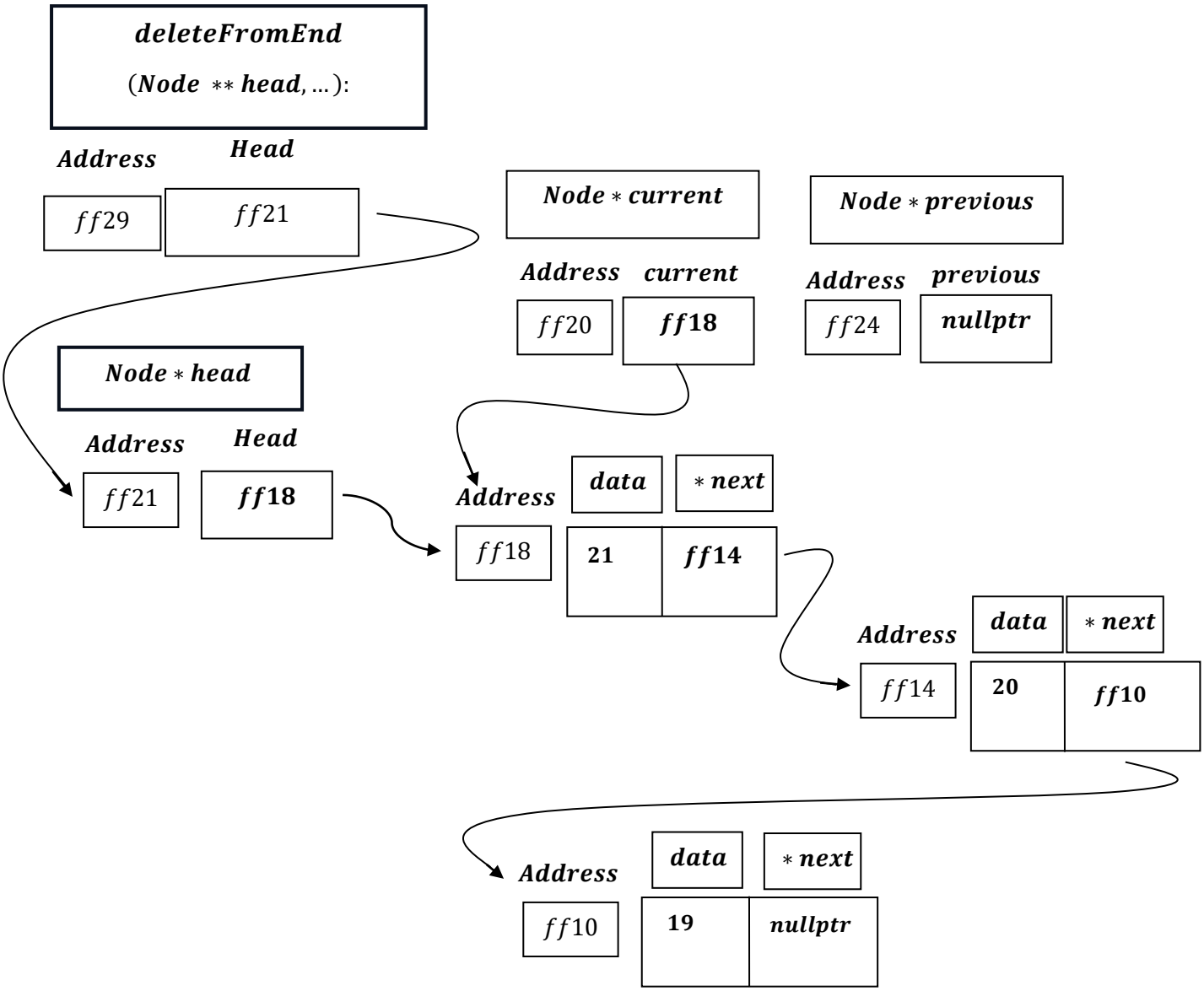
```
    else
    {
        Node *current = *head;
        Node *previous = nullptr;

        while (current->next != nullptr)
        {
            previous = current;
            current = current->next;
        }
        if (previous == nullptr)
        {
            *head = nullptr;
        }
        else
        {
            previous->next = nullptr;
        }
        cout << "Deleted node: " << current->data << endl;
        free(current);
        current = nullptr;
        previous = nullptr;
        cout << "Node is deleted successfully." << endl;
    }
}
```

At first:

```
        Node *current = *head;
```




```
Node *previous = nullptr;
```



Now,

```
        while (current->next != nullptr)
         {
             previous = current;
             current = current->next;
         }
```

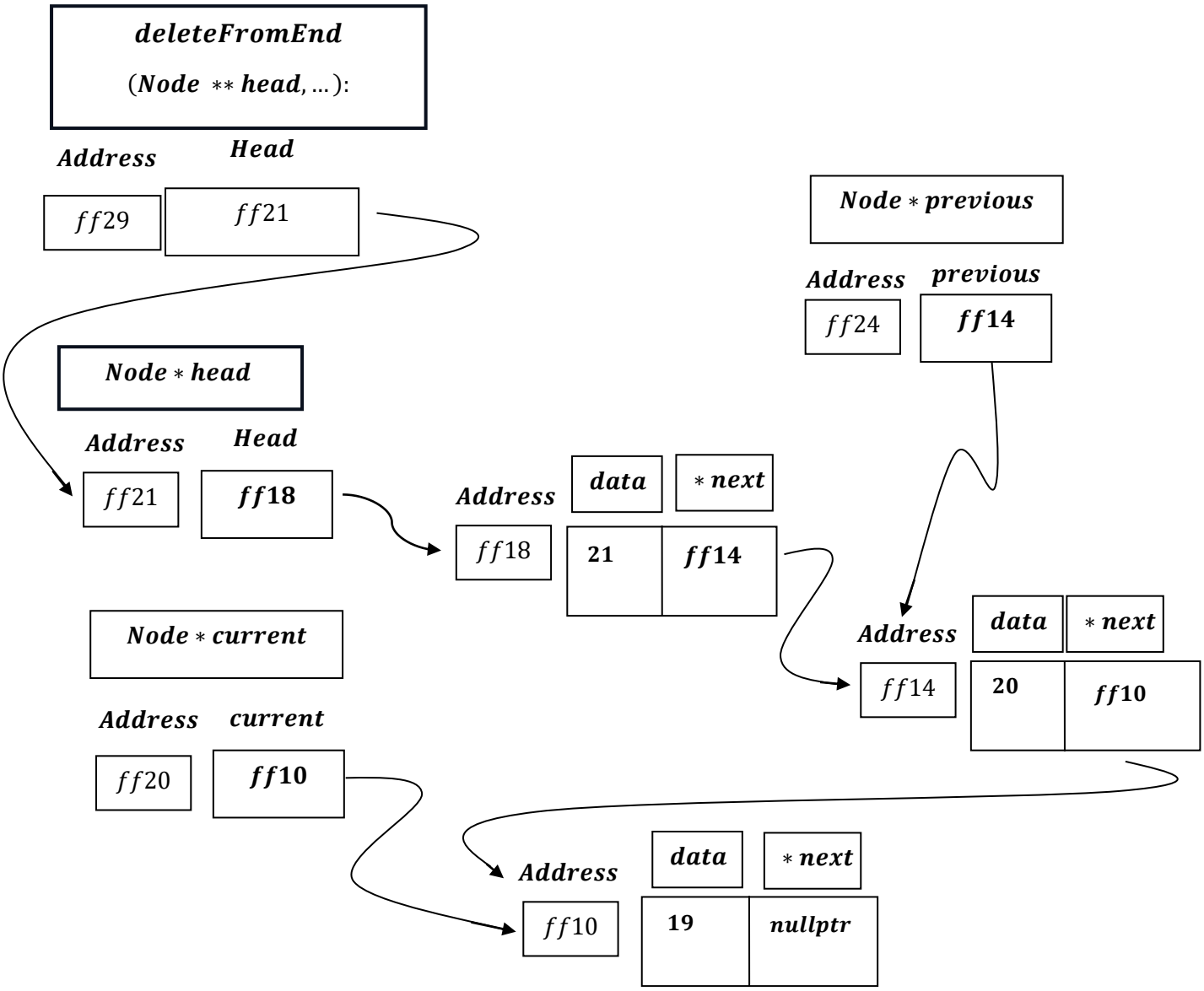
Till next pointer will not become nullpointer , loop will continue , i.e. if next pointer equals to nullpointer , loop will get terminated.

$Hence\ , during\ the\ loop\ terminates\ , previous\ and\ current\ will\ be\ at$:

$1st\ \ time\ loop\ runs\ then\ , previous = ff18\ and\ current = current \rightarrow next = ff14.$

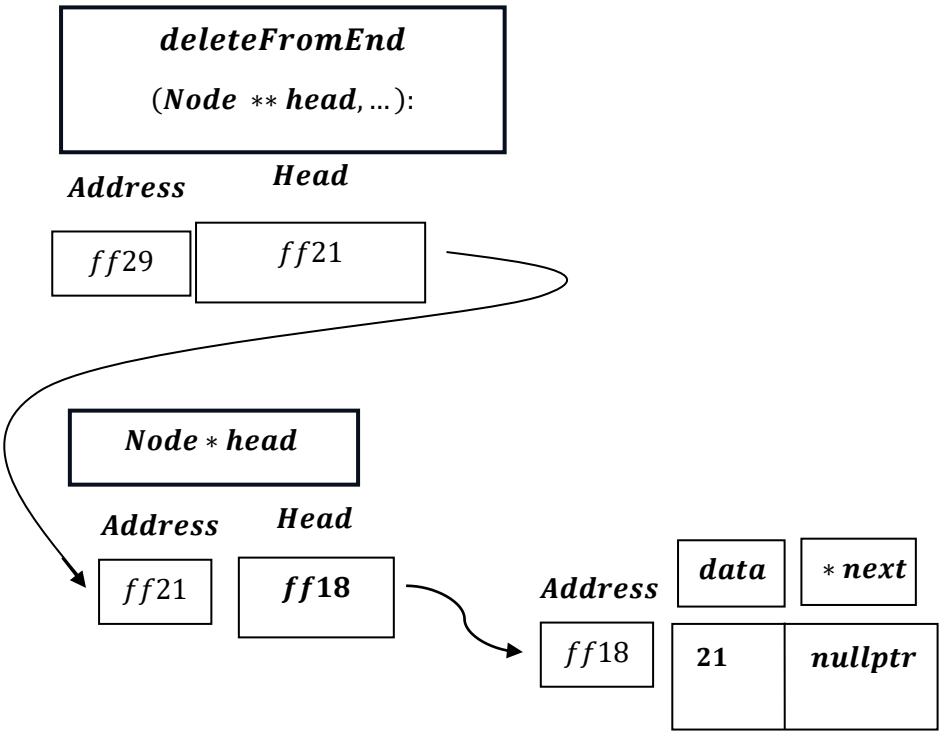
$2nd\ time\ loop\ runs\ then, previous = ff14\ and\ current = current \rightarrow next = ff10.$

$3rd\ time\ , current = current \rightarrow next = nullptr, hence\ loop\ terminates.$



Now , there is two common scenarios :

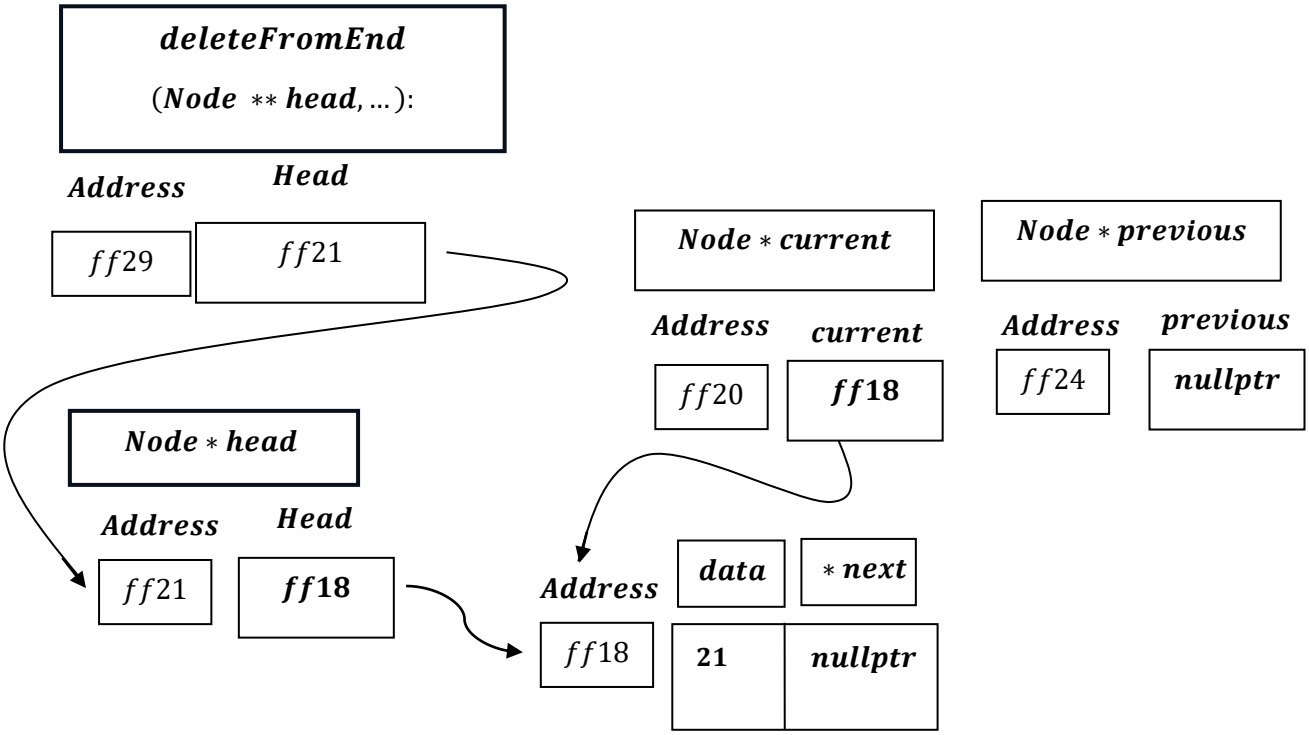
1) *Head pointer is pointing at single head node*:



And,

```
Node *current = *head;
Node *previous = nullptr;
```

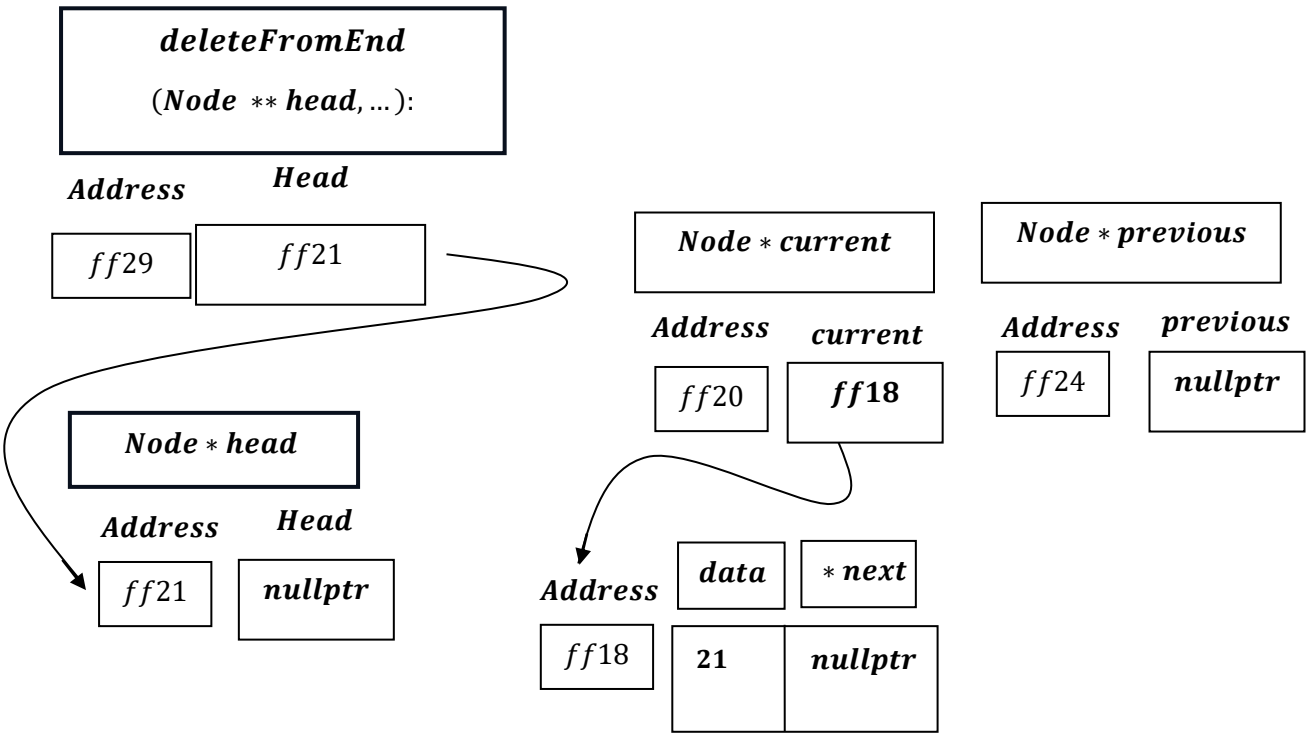
i.e.,



Therefore at this type of scenarios , loop doesnot executes as at first attempt only , current → next = null. And previous will remain null, this indicates , the linked list have single node.

Therefore,

```
if (previous == nullptr)

        {
            *head = nullptr;


        }
```



As, head pointer is nullptr now , it starts pointing to address `ff18` .

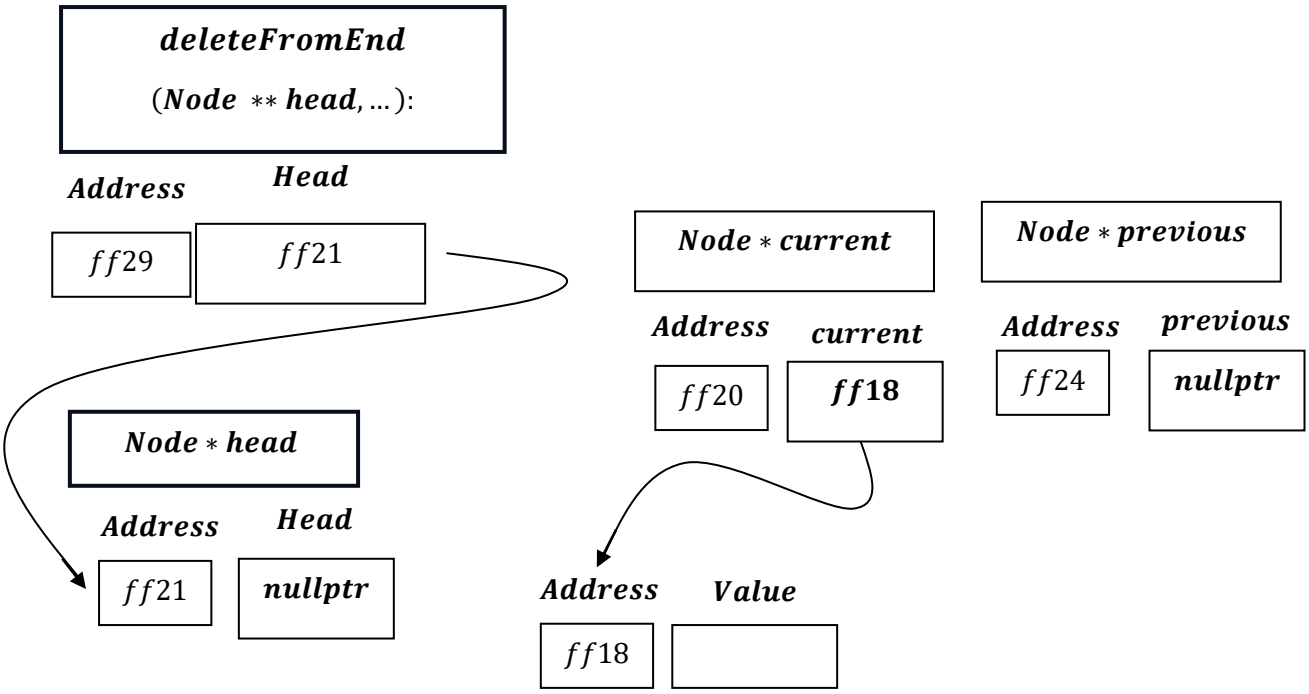
```
        cout << "Deleted node: " << current->data << endl;
```

Next, we print out the data that contains in the node which is to be deleted.

Now we have to free current, which means current pointer will still be pointing to address: ff18 , but memory pointed by current pointer will get freed.

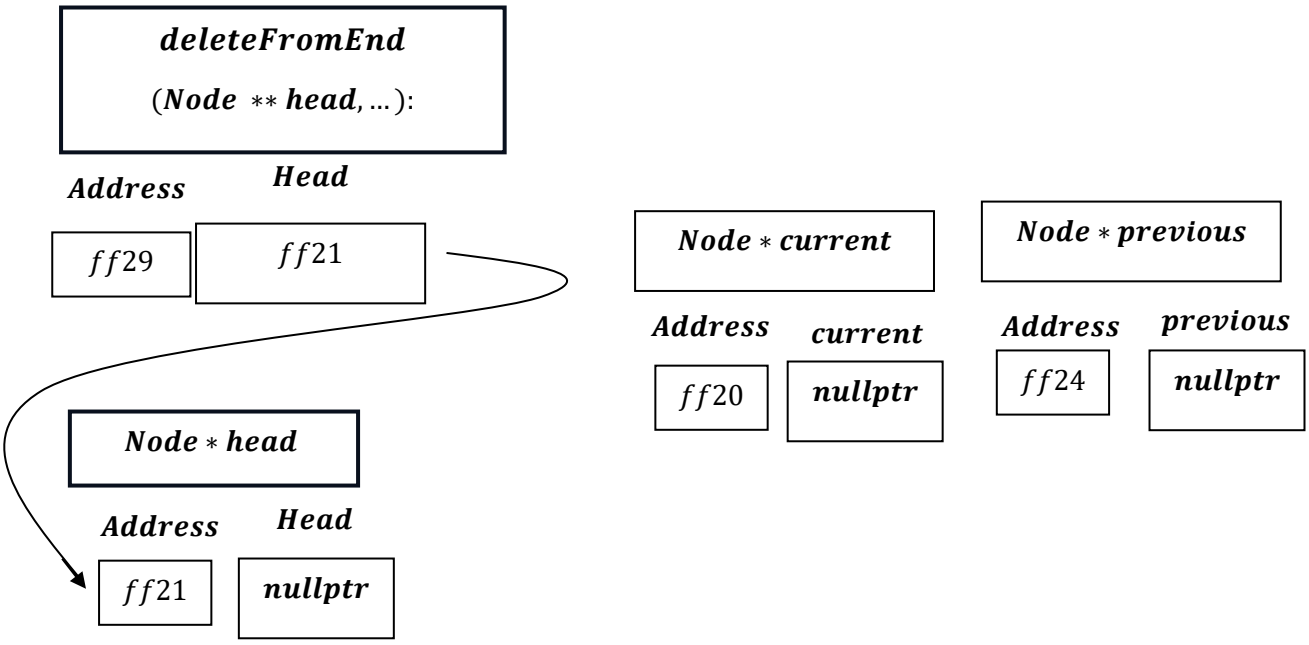
As soon as we free the memory pointed to by current, both the data and the next pointer associated with that node are deallocated. The free(current) call releases the memory, making it available for reuse, but it also means that the entire node (including data and next pointer) is no longer accessible.

```
        free(current);
```



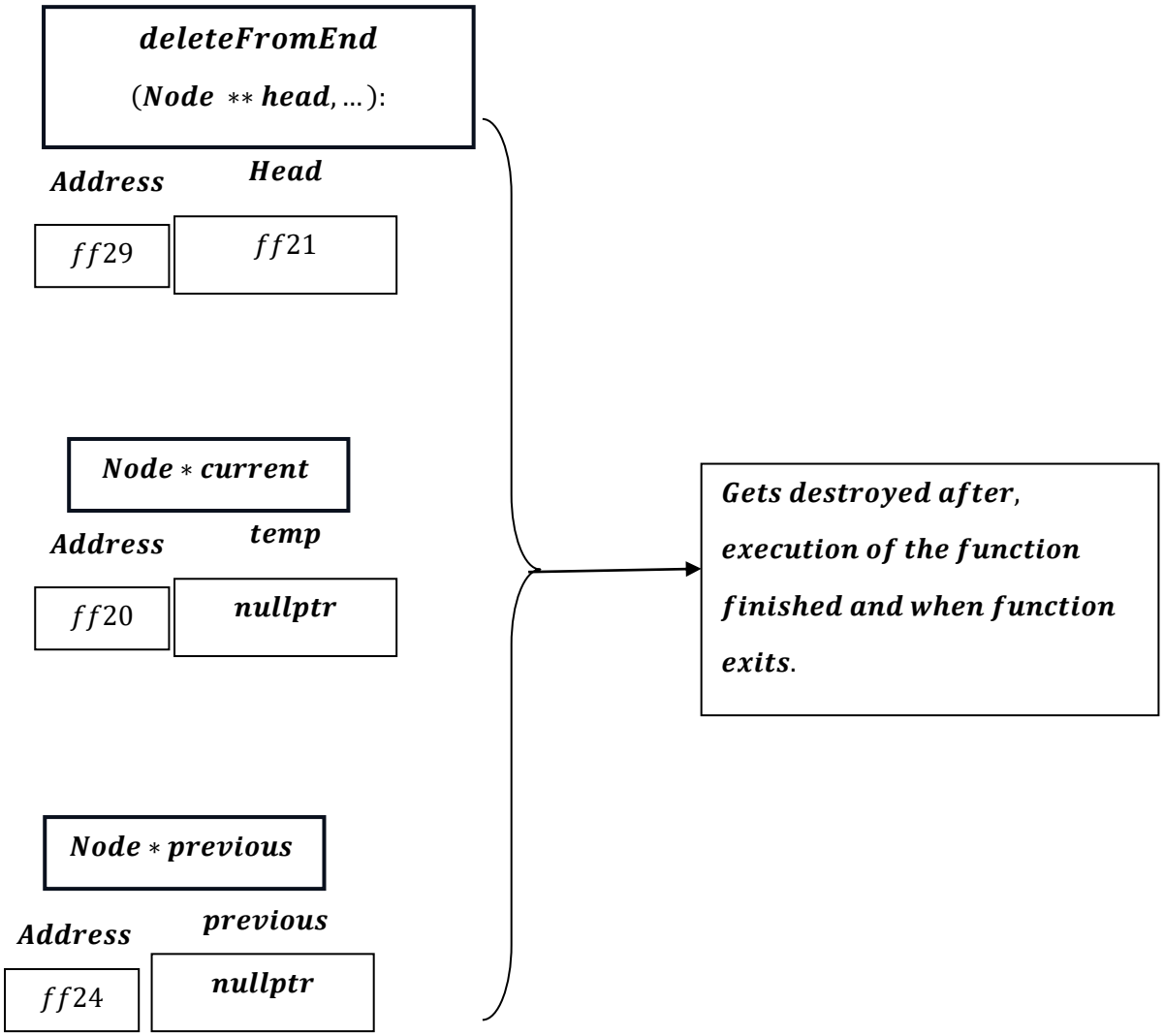
Now we set current pointer to null to avoid dangling pointers.

```
current = nullptr;
```

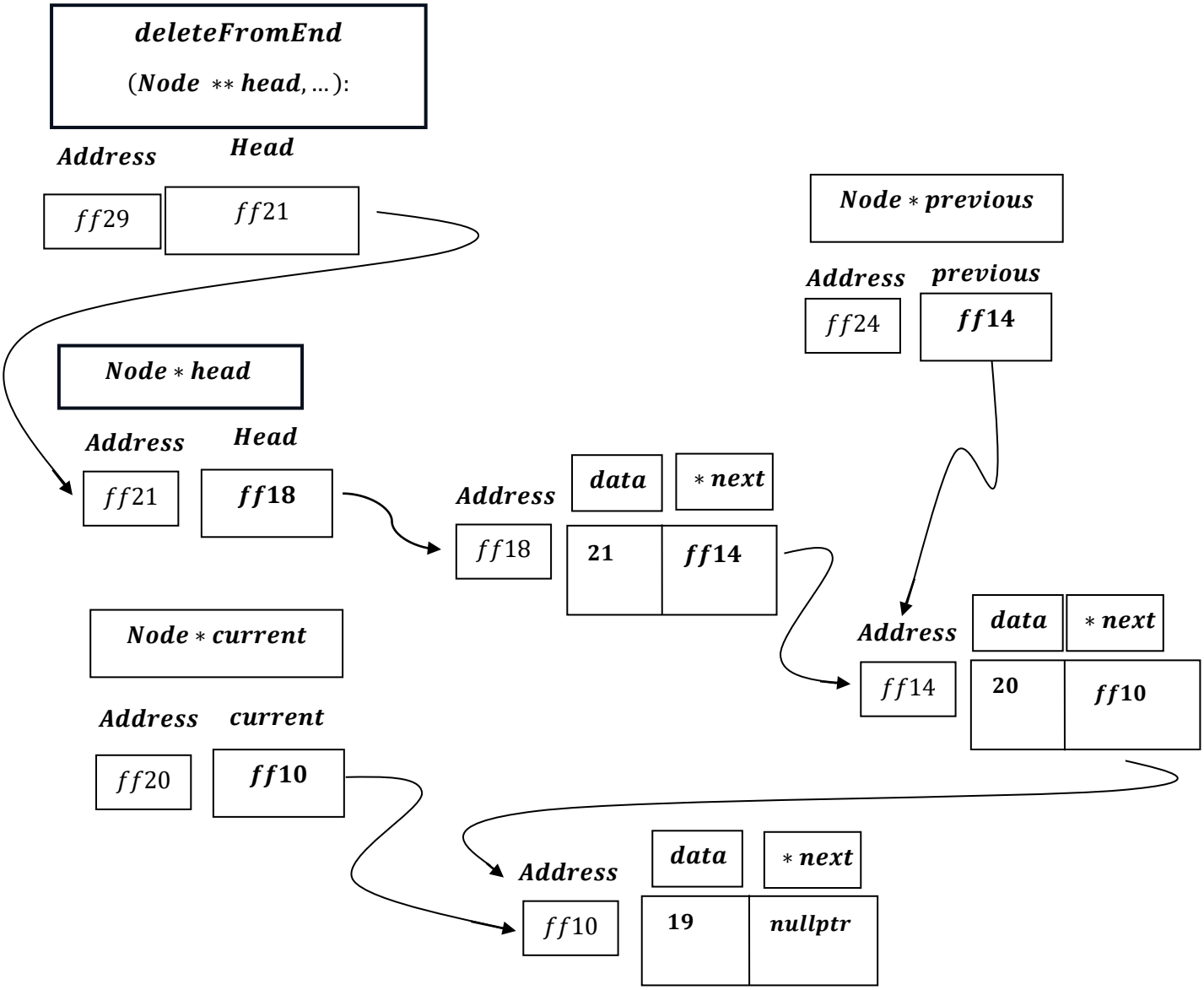


After the function module ends , all temporary pointer variables created gets destroyed.

i.e.

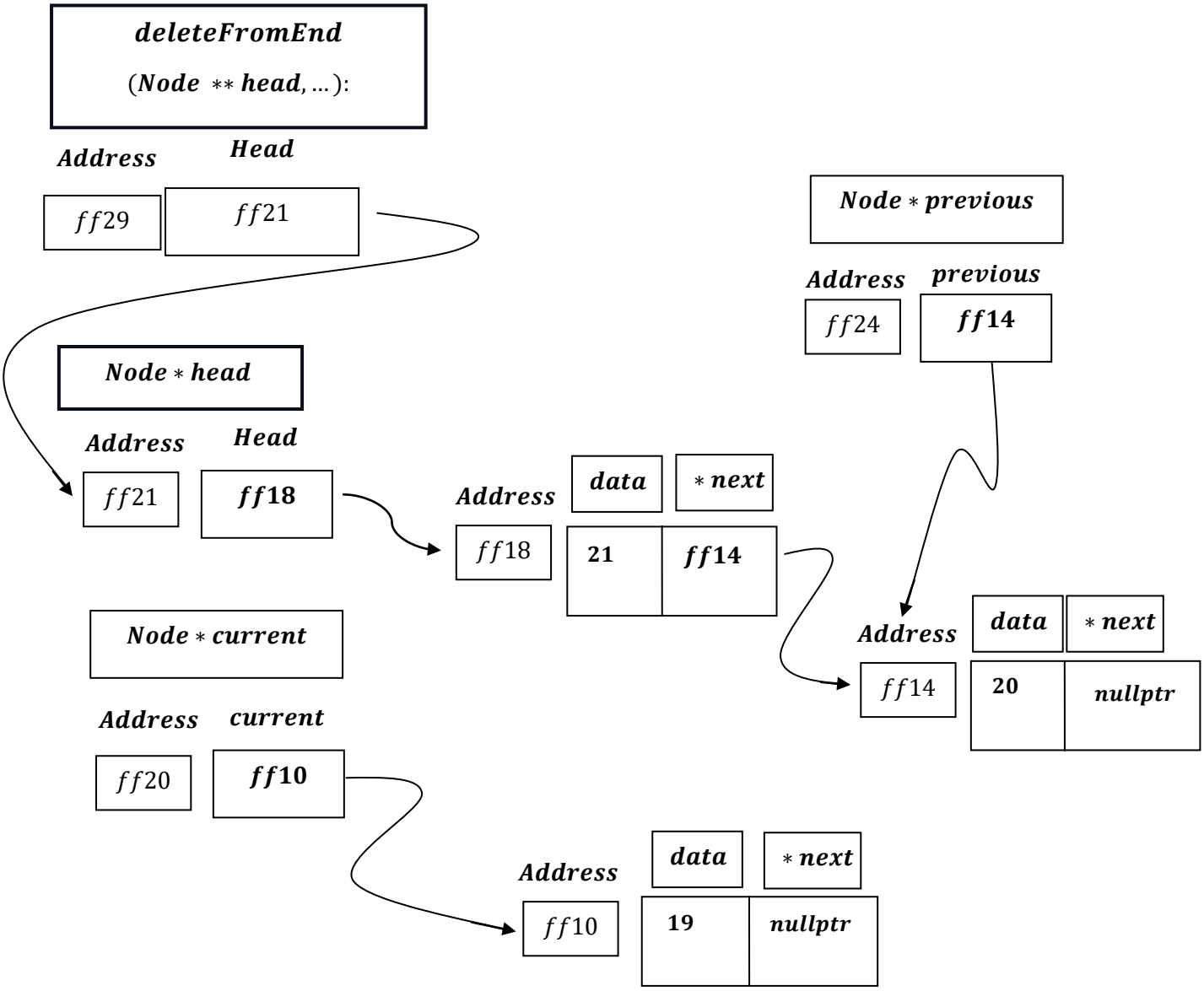


2)*We have Linked List with numerous nodes with a head node and a head pointer pointing to the head*:



***Next carrrying forward from earlier , now as previous pointer doesnot have null pointer , hence*:**

```
    else
      {
          previous->next = nullptr;
      }
```



As, previous → next = nullptr; the next pointer pointed by previous pointer , the next pointer stops pointing to any memory location.

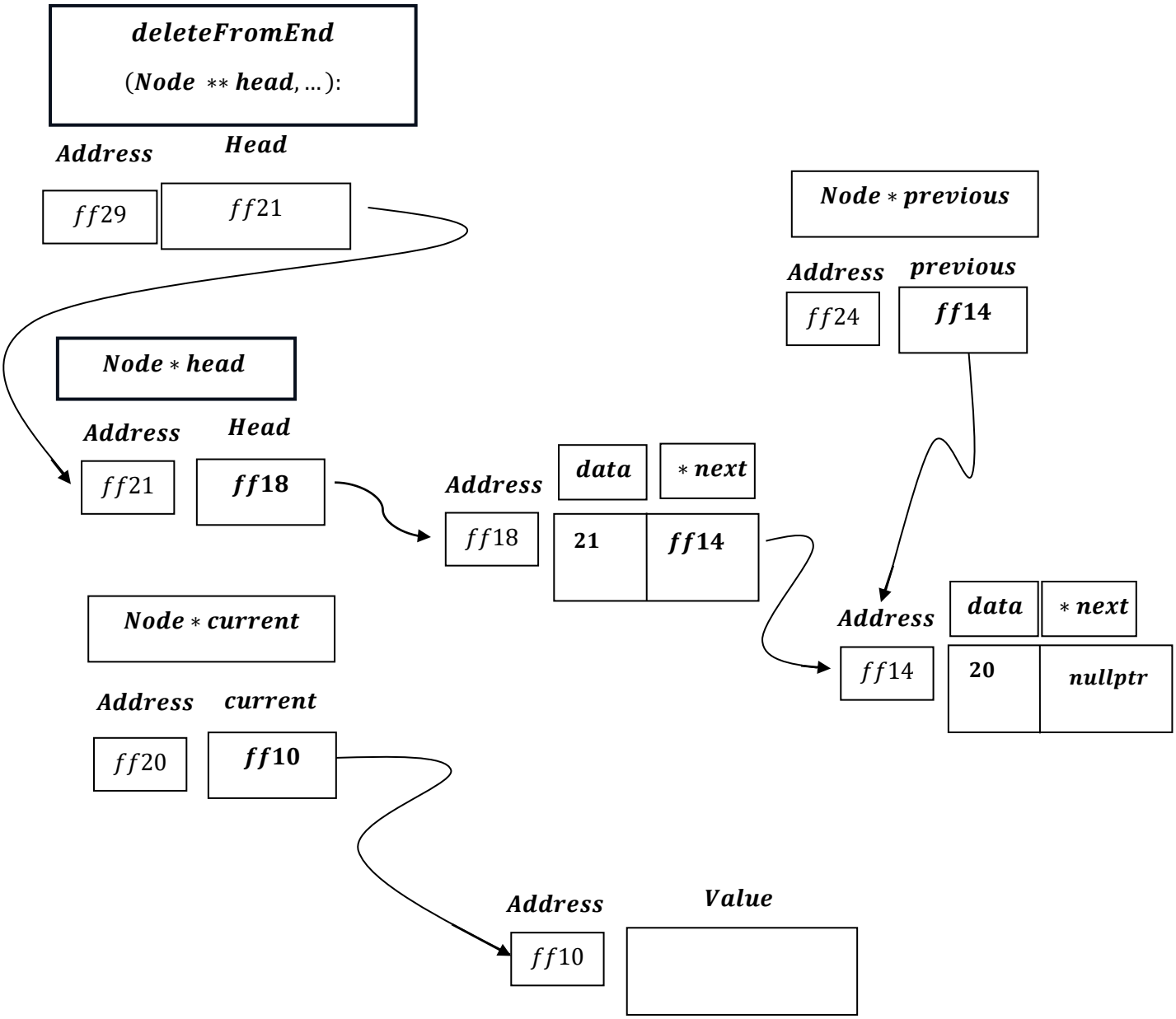
```
cout << "Deleted node: " << current->data << endl;
```

Next, we print out the data that contains in the node which is to be deleted.

```
free(current);
```

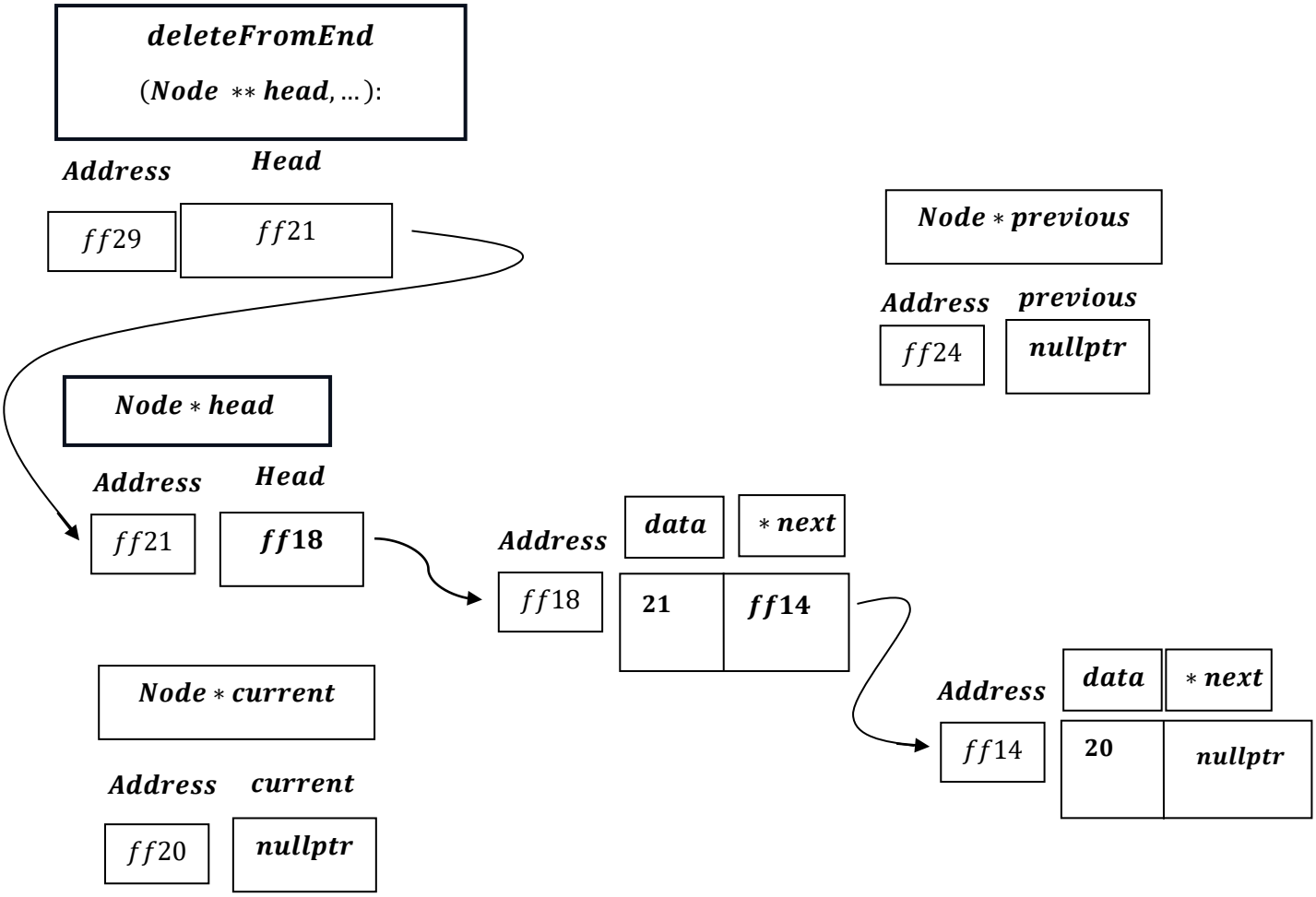
Now we have to free current, which means allocated memory pointed by current pointer will set free. Current pointer will still be pointing to , holding memory location : ff10.

As soon as we free the memory pointed to by current, both the data and the next pointer associated with that node are deallocated. The free(current) call releases the memory, making it available for reuse, but it also means that the entire node (including data and next pointer) is no longer accessible.



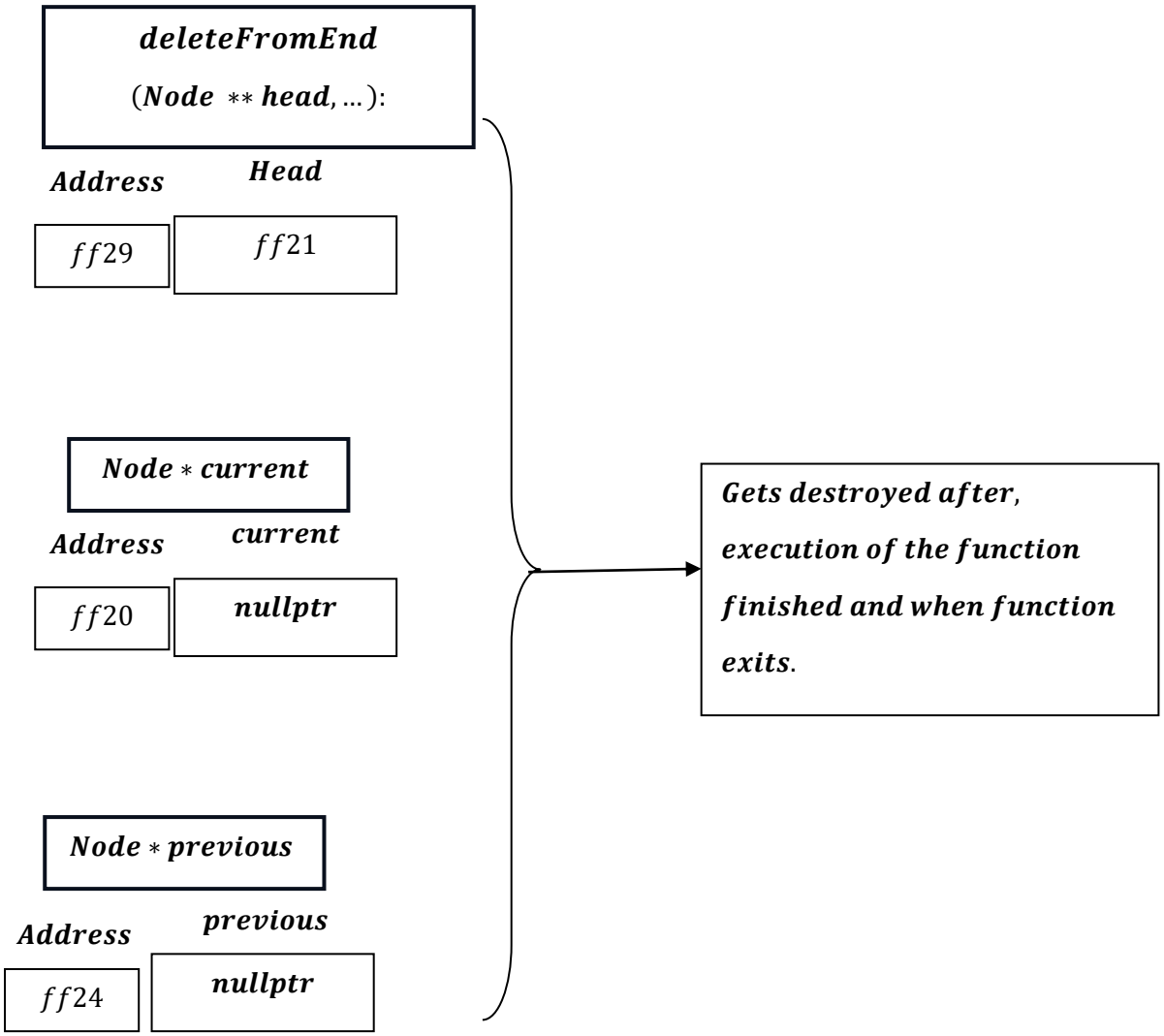
Now , to avoid dangling pointer we set current and previous pointer to nullptr.

```
current = nullptr;
previous = nullptr;
```



After the function module ends , all temporary pointer variables created gets destroyed.

i.e.



The deleteFromEnd function step by step:

1. ***Check if the List is Empty***:

−If the head pointer is nullptr, it means the list is empty. The function prints an error message and returns immediately.

2. *Initialize Pointers*:

−current is initialized to point to the head node.
−previous is initialized to nullptr. This will be used to keep track of the node before current.

3. *Traverse to the Last Node*:

−The loop continues until current points to the last node (i.e., the node whose next pointer is nullptr).

−previous is updated to point to the node before current in each iteration.

4. *Handle Deletion*:

−If previous is still nullptr after the loop, it means the list had only one node. In this case, the head pointer is set to nullptr.

−Otherwise, the next pointer of previous is set to nullptr, effectively removing the last node from the list.

5. *Print Deleted Node Data and Free Memory*:

−The data of the deleted node is printed.
−The memory allocated for the current node is freed.
−A success message is printed.

Time Complexity Of Delete From End

```
void deleteFromEnd(Node **head)
{
    if (*head == nullptr)                                   O(1)
    {
        cout << "Error: List is empty." << endl;            O(1)
        return;                                             O(1)
    }
    else
    {
        Node *current = *head;                              O(1)
        Node *previous = nullptr;                           O(1)

        while (current->next != nullptr)                    runs `n − 1` times.
        {
            previous = current;                             1 + 1 + ⋯ (n − 1) times = O(n) times.
            current = current->next;                        1 + 1 + ⋯ (n − 1)times = O(n) times.
        }
        if (previous == nullptr)                            O(1)
        {
            *head = nullptr;                                O(1)
        }
        else
        {
            previous->next = nullptr;                       O(1)
        }
        cout << "Deleted node: " << current->data << endl;  O(1)
        free(current);                                      O(1)
        current = nullptr;                                  O(1)
        previous = nullptr;                                 O(1)
        cout << "Node is deleted successfully." << endl;    O(1)
    }
}
```

$1.$ ***Best Case , when there is single head node , where head pointer pointing to the single head node or List is Empty.***

<u>$1.$ ***When List is Empty***:</u>

$\rightarrow$ ***If*** $*$ ***head*** $==$ ***nullptr , this checking takes constant time. The cout and return statements takes constant time. Hence overall process takes,*** $O(1)$ ***time.***

<u>$2.$ ***When List has a single Node***:</u>

$\rightarrow$ ***current pointer's value gets assigned to head pointer's value. This assignment takes constant time i.e.*** $O(1)$.

$\rightarrow$ ***previous pointer value is assigned to `nullptr` . T his assignment also takes constant time i.e.*** $O(1)$.

$\rightarrow$ ***Now, if there is single node then previous pointer will remain assigned to nullpointer, hence if*** $(previous == nullptr)$ ***, this checking takes constant amount of time . Then head pointer which was assigned to the single head node's address, will now get assigned to `nullptr`, this takes contant amount of time. Hence overall time complexity is:*** $O(1)$.

$\rightarrow$ ***Now as current is still pointed to the memory location, where data to be deleted is present , hence we print the deleted data which takes constant amount of time i.e.*** $O(1)$.

$\rightarrow$ ***Now we free the memory pointed by current pointer which again takes constant amount time i.e.*** $O(1)$.

$\rightarrow$ *Assigning current pointer's value to nullptr to avoid dangling pointer which takes constant amount time i.e.* $O(1)$.

$\rightarrow$ *Now, a message gets printed that , deletion have taken place successfully takes another contant amount of time i.e.* $O(1)$.

Hence , Overall time complexity for Best Case is $O(1)$.

---

2. *Worst Case , when there present multiple nodes in the linked list.*

$\rightarrow$ *current pointer's value gets assigned to head pointer's value. This assignment takes constant time i.e.* $O(1)$.

$\rightarrow$ *previous pointer value is assigned to `nullptr` . T his assignment also takes constant time i.e.* $O(1)$.

$\rightarrow$ *Now till next pointer doesnot become nullptr , previous pointer gets assigned to current pointer and current pointer gets assigned to next pointer value. This loop runs one less than the node `n` i.e.* $n-1$ *times , and the inner statements* :

$previous = current\ runs: 1+1+1+\cdots(n-1)\ times = n-1.$

$current = current \rightarrow next\ runs : 1+1+1+\cdots(n-1) times = n-1.$

$Therefore\ overall\ calculation = \ 2n-2 = 2(n-1)$

Applying assymptotic big O in the equation:

$= O\big(2(n-1)\big)$

$= 2\big(O\ (n) - O(1)\big)\ [O(n)\ dominates\ over\ O(1)]$

$= 2 \times O(n)$ $[c \times O(n) = O(n), constant\ factor\ is\ ignored\ ]$

$= O(n)$

$Hence, time\ complexity\ takes\ is : O(n).$

$\rightarrow As, previous\ doesnot\ hold\ value\ nullptr\ now, hence\ next\ pointer\ pointed\ by\ previous\ pointer\ of\ structure\ Node\ gets\ assigned\ to\ nullptr, to\ deallocate\ itself\ from\ the\ last\ node\ takes\ constant\ time\ i.e. O(1)$.

$\rightarrow Now\ as\ current\ is\ still\ pointed\ to\ the\ memory\ location, where\ data\ to\ be\ deleted\ is\ present\ i.e. the\ last\ node, hence\ we\ print\ the\ deleted\ data\ which\ takes\ constant\ amount\ of\ time\ i.e. O(1).$

$\rightarrow$ $Now\ we\ free\ the\ memory\ pointed\ by\ current\ pointer\ which\ again\ takes\ constant\ amount\ time\ i.e. O(1).$

$\rightarrow Assigning\ current\ pointer's\ value\ to\ nullptr\ to\ avoid\ dangling\ pointer\ which\ takes\ constant\ amount\ time\ i.e. O(1).$

$\rightarrow Now, a\ message\ gets\ printed\ that\ , deletion\ have\ taken\ place\ successfully\ takes\ another\ contant\ amount\ of\ time\ i.e. O(1).$

$$Hence\ , Overall\ time\ complexity\ for\ Worst\ Case\ is:$$

$$O(1) + O(1) + O(n) + O(1) + O(1) + O(1) + O(1) + O(1) = O(n).$$

We can say , if List is Empty , or we have a single head node connected to Linked List, we get our best case scenario, and when list is not empty and there present multiple nodes , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the deletion to be made , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

In the case of DeleteAtEnd, the deletion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the deletion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence DeleteAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$ *, we get:*

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$

Singly Linked List	***Cases***	***Time Complexity***
DeleteAt End	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****